37.4 Nix 声明式部署

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~220k tokens，输出 ~5k tokens（本节）

OpenClaw 官方维护了一个 Nix 生态集成项目 **nix-openclaw**，提供了基于 Home Manager 的声明式部署方案。与 npm 全局安装或 Docker 容器不同，Nix 部署具有**可复现性**、**即时回滚**和**声明式配置管理**三大优势。

> **衍生解释**：Nix 是一个函数式包管理器和构建系统。它的核心思想是将软件包的构建过程视为纯函数——相同的输入（源码 + 依赖）必然产生相同的输出（构建产物），这保证了可复现性。NixOS 将这一理念扩展到整个操作系统，而 Home Manager 则专注于用户级的程序和配置管理。**Flake** 是 Nix 的现代项目格式，通过 `flake.nix` 和 `flake.lock` 精确锁定所有依赖版本。

37.4.1 nix-openclaw 概述

nix-openclaw 是一个 Nix Flake 项目，提供：

- **Home Manager 模块**：声明式配置 Gateway、macOS 应用、工具链
- **Launchd/systemd 服务**：自动生成守护进程配置
- **插件系统**：声明式管理 OpenClaw 扩展
- **模板**：快速上手的 flake 模板

部署架构

```
flake.nix（用户配置）
    │
    ├── inputs.nix-openclaw（Flake 输入）
    │
    └── home-manager.modules = [{
            programs.openclaw = {
                enable = true;
                gateway.enable = true;
                gateway.port = 18789;
                gateway.auth.mode = "token";
                # ...
            };
        }]
            │
            ├── 生成 openclaw.json（配置文件）
            ├── 生成 launchd plist / systemd unit
            ├── 安装 OpenClaw npm 包（固定版本）
            └── 设置 OPENCLAW_NIX_MODE=1
```

37.4.2 快速上手

nix-openclaw 提供了一种面向 AI Agent 的安装模式——用户可以直接将安装指令发送给 AI 助手（如 Claude、Cursor），让 AI Agent 完成全部配置工作：

```
I want to set up nix-openclaw on my Mac.
Repository: github:openclaw/nix-openclaw

What I need you to do:
1. Check if Determinate Nix is installed (if not, install it)
2. Create a local flake at ~/code/openclaw-local using templates/agent-first/flake.nix
3. Set up secrets (API keys) - plain files at ~/.secrets/ is fine
4. Fill in the template placeholders and run home-manager switch
5. Verify: launchd running, gateway responds
```

手动安装步骤：

```
# 1. 安装 Determinate Nix（如果尚未安装）
curl --proto '=https' --tlsv1.2 -sSf -L \
  https://install.determinate.systems/nix | sh -s -- install

# 2. 从模板创建项目
nix flake init -t github:openclaw/nix-openclaw#agent-first
# 或克隆到指定目录
mkdir -p ~/code/openclaw-local && cd ~/code/openclaw-local
nix flake init -t github:openclaw/nix-openclaw#agent-first

# 3. 编辑 flake.nix，填写配置
# 4. 应用配置
home-manager switch --flake .
```

37.4.3 Nix Mode 运行时行为

当 OpenClaw 在 Nix 环境下运行时， `nix-openclaw` 会自动设置环境变量 `OPENCLAW_NIX_MODE=1` 。OpenClaw 核心通过 `resolveIsNixMode()` 检测此模式：

```
// src/config/paths.ts
export function resolveIsNixMode(env = process.env): boolean {
  return env.OPENCLAW_NIX_MODE === "1";
}
```

Nix 模式下的行为变化：

特性	普通模式	Nix 模式
自动安装流程	启用	**禁用**
依赖缺失提示	通用安装指引	**Nix 特定修复建议**
控制台 UI	正常	显示只读 Nix 模式横幅
配置路径	`~/.openclaw/openclaw.json`	由 Nix 管理（只读）
状态目录	`~/.openclaw/`	`OPENCLAW_STATE_DIR` 指向 Nix 管理路径

路径管理

在 Nix 模式下，配置文件和状态目录通过环境变量精确控制：

```
# 由 nix-openclaw 自动设置
OPENCLAW_NIX_MODE=1
OPENCLAW_CONFIG_PATH=/path/to/nix-managed/openclaw.json   # 只读配置
OPENCLAW_STATE_DIR=~/.openclaw                             # 可变状态
```

`resolveStateDir()` 和 `resolveConfigPath()` 优先读取环境变量覆盖值：

```
export function resolveStateDir(env = process.env): string {
  const override = env.OPENCLAW_STATE_DIR?.trim();
  if (override) return resolveUserPath(override);
  // ... 回退逻辑
}
```

这种设计确保了 Nix 的不可变存储（Nix store）中的配置文件不会被运行时修改，同时可变状态（会话、日志、缓存）仍然可以正常写入。

macOS 应用集成

macOS 应用不会自动继承 shell 环境变量，因此 Nix 模式也可以通过 macOS defaults 命令设置：

```
defaults write bot.molt.mac openclaw.nixMode -bool true
```

37.4.4 声明式配置示例

一个完整的 `flake.nix` 配置示例：

```nix
{
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixpkgs-unstable";
    home-manager.url = "github:nix-community/home-manager";
    nix-openclaw.url = "github:openclaw/nix-openclaw";
  };

  outputs = { nixpkgs, home-manager, nix-openclaw, ... }: {
    homeConfigurations."myuser" = home-manager.lib.homeManagerConfiguration {
      pkgs = nixpkgs.legacyPackages.aarch64-darwin;
      modules = [
        nix-openclaw.homeManagerModules.default
        {
          programs.openclaw = {
            enable = true;

            gateway = {
              enable = true;
              port = 18789;
              bind = "loopback";

              auth = {
                mode = "token";
                # token 从 secrets 文件读取
              };

              tailscale = {
                mode = "serve";
                resetOnExit = true;
              };

              discovery.mdns.mode = "minimal";
            };

            # API 密钥通过 secrets 管理
            secrets.anthropicKey = "/Users/myuser/.secrets/anthropic-key";
          };
        }
      ];
    };
  };
}
```

37.4.5 与其他部署方式的对比

维度	npm 全局	Docker	Nix
安装方式	`npm i -g`	`docker compose up`	`home-manager switch`
可复现性	✗（依赖宿主 Node）	△（依赖镜像版本）	✓（精确锁定）
回滚	手动	`docker pull` 旧版	`home-manager switch --rollback`
配置管理	手动编辑 JSON	环境变量 + 挂载	声明式 Nix 表达式
隔离性	无	容器隔离	Nix store 隔离
macOS 集成	✓（原生）	△（需端口映射）	✓（launchd + 原生应用）
适用场景	快速上手	服务器部署	高级用户 / 团队标准化

Nix 部署的核心优势在于**声明式可复现**——整个系统的状态（包括 OpenClaw 版本、配置、依赖）都在一个 `flake.nix` 文件中精确描述，任何人在任何时间用同一个 flake 都能得到完全相同的结果。

本节小结

1. **nix-openclaw** 是 OpenClaw 的官方 Nix 集成，以 Home Manager 模块形式提供声明式部署。
2. **Nix 模式**（`OPENCLAW_NIX_MODE=1`）改变运行时行为：禁用自动安装流程、显示 Nix 特定的错误提示、配置路径由环境变量精确控制。
3. **路径分离设计**确保 Nix 的不可变配置（只读）和运行时可变状态（会话、缓存）互不干扰。
4. 相比 npm 和 Docker，Nix 提供精确可复现性和即时回滚能力，尤其适合高级用户和团队标准化部署。
5. Nix 部署方案体现了 OpenClaw 对多种部署哲学的支持——从最简单的 npm 全局安装到最严格的声明式管理，用户可以根据需求选择。